Computer Vision in Helping Eyes

Every picture, pixel and bit of math in this project, in very simple words.

Almost all of the computer vision lives in one file: helping_eyes/vision.py. It uses **OpenCV** and **NumPy** only. The one exception is reading the letters: that is done by **Apple Vision**, the text reader built into macOS. There is no model to train and no GPU. Everything below is "classic" computer vision: small, clear steps you can follow with a pencil.

Contents

0. The big picture
1. What a picture really is
2. The small tools we use again and again
3. Part A: Reading the text (OCR)
4. Part B: Is the picture good enough? (quality)
5. Part B: Finding the page and flattening it (homography)
6. Part B: Straightening curved lines (dewarp)
7. Part B: Fixing a slanted page (skew)
8. Part B: Fixing shadows and bad light
9. Part C: Book mode, when to read (page turns)
10. Part C: Book mode, where the pages are (the spine)
11. Part C: Page number, title, header, footer
12. Part C: In what order to read (XY-cut)
13. Part C: Putting the whole page together
14. Part C: Carrying on after the view moves
15. Part D: Live guidance ("Move closer", "Hold still")
16. Part D: The book-mode decision
17. The glare experiment (glare.py)
18. Cheat sheet: every number and why
19. Math glossary
20. How the checks did on 148 real photos
21. Book mode and live checks on real input

---

0. The big picture

A camera frame goes in. Spoken guidance and readable text come out.

```
browser camera (JPEG)
       |
       v
server decodes it into a grid of numbers (a NumPy array)
       |
       +--> PART D  live guidance: "is there text? is it in view? hold still..."
       |         uses  PART A (find text)  and  PART B (quality)
       |
       +--> BOOK MODE  PART C: "when to read, where the pages are, what order"
                 uses  PART B (flatten / straighten)  and  PART A (read letters)
```

The four parts:

Part	Question it answers	Main functions
A	What does the text say, and where is each line and word?	`recognize_text`, `find_text_region`, `read_text_enhanced`
B	Is the picture good? Can we make it better?	`assess_quality`, `find_page_quad`, `page_fill`, `warp_page`, `estimate_dewarp`, `estimate_skew`, `enhance_tone`, `binarize`
C	Book mode: when, where, in what order?	`PageTurnDetector`, `find_book_quad`, `find_spine`, `split_spread`, `split_page_parts`, `xy_cut`, `read_page`, `continue_from`
D	Live coaching and the final decision	`LiveGuide`, `BookWatcher`, `BookReader`

Each part uses the ones before it: D uses C, C uses B, and all use A.

Where the work happens. The browser (web/app.js) only *draws and speaks*. It asks the camera for about 1920×1080 (`getUserMedia`), shrinks the frame onto a hidden canvas, and sends it as a **JPEG** to the server. The server turns the JPEG back into numbers with `cv2.imdecode` and makes every decision. Boxes sent back to the browser are **normalised to 0–1**, so they work at any screen size.

---

1. What a picture really is

1.1 Pixels

A digital picture is a **grid of tiny squares called pixels**. Each pixel is just a number (or three numbers). A 1920×1080 picture has 1920 columns and 1080 rows, so 2,073,600 pixels.

In NumPy a picture is an array:

```
image.shape == (height, width, 3)      # rows, columns, colour channels
image[y, x]  == [B, G, R]              # the three numbers of one pixel
```

Two things that surprise people:

1. **The order is B, G, R, not R, G, B.** That is how OpenCV stores colour. (The code says "BGR image" everywhere.)
2. **Row first, column second.** `image[y, x]`, so `y` (down) comes before `x` (across). Shape is `(h, w)`, written height first.

1.2 Brightness numbers (0 to 255)

Each colour channel is one byte (`uint8`): **0 = none, 255 = full**.

```
pixel [0, 0, 0]       = black
pixel [255, 255, 255] = white
pixel [0, 0, 255]     = pure red   (B=0, G=0, R=255)
```

1.3 Greyscale

For most of the work we do not need colour. We squash the three numbers into one "how bright" number, using how much the human eye likes each colour (green counts most):

```
grey = 0.299 × R + 0.587 × G + 0.114 × B
```

That is `cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)`. Now one number per pixel, 0 (black) to 255 (white). **Black ink on white paper** becomes "low numbers on high numbers", and that contrast is what everything else hunts for.

1.4 Coordinates

- Origin `(0, 0)` is the **top-left** corner. `x` grows to the right, `y` grows **downwards**.
- A **box** in this project is `(x1, y1, x2, y2)`: top-left corner, bottom-right corner.
- **Apple Vision** is different: its origin is the **bottom-left** and its numbers are fractions (0–1) of the picture. `_vision_box` flips it:

```
x1 = x × width
y1 = (1 − y − box_height) × height      ← the "1 −" flips bottom-up into top-down
x2 = (x + box_width) × width
y2 = (1 − y) × height
```

- **Normalising** (`_norm`) divides by the picture size so a box becomes fractions of 0–1: `x / width`, `y / height`. The browser multiplies by *its* canvas size to draw it.

1.5 Resizing, and why we shrink pictures

Big pictures are slow and noisy. Many steps first shrink the picture (for example to 640 or 480 pixels wide). `cv2.resize(..., interpolation=cv2.INTER_AREA)` shrinks by **averaging the pixels that fall into each new pixel**, which is the cleanest way to make an image smaller. Measuring things (like sharpness) at a *fixed* width also means the answer does not depend on which camera you have.

1.6 Noise

Real camera pixels are never perfectly steady. Even a still scene wobbles by a few grey levels. That wobble is **noise**. Several parts of the code take care not to mistake noise for real change.

---

2. The small tools we use again and again

Learn these once and the rest reads easily.

2.1 Blur (Gaussian)

Replace every pixel by a **weighted average of its neighbours**, nearer ones counting more. The weights follow the bell curve:

```
G(x, y) = 1 / (2π σ²) · e^( −(x² + y²) / (2σ²) )
```

Effect: tiny specks (noise) melt away and edges get soft. We blur *before* other steps so noise does not fool them. `cv2.GaussianBlur(img, (7, 7), 0)` means "use a 7×7 neighbourhood".

2.2 Convolution (sliding a small window)

Blur, edge finding and many others are one idea: **slide a small grid of weights (a kernel) over the picture; at each spot multiply and add.** Different weights, different effect.

```
Average (blur):          Laplacian (edges):
 1/9 1/9 1/9              0   1   0
 1/9 1/9 1/9              1  −4   1
 1/9 1/9 1/9              0   1   0
```

2.3 Thresholding (black or white, nothing between)

Pick a cut-off *t*. Every pixel brighter than *t* becomes white (255), the rest become black (0). That turns a grey picture into a **mask**: a yes/no map of "this pixel is interesting".

- **Fixed threshold**: you choose *t* (for example "glare = at least 250").
- **Otsu's threshold** (`THRESH_OTSU`): the computer chooses *t* itself. It tries every *t* from 0 to 255 and keeps the one that best splits the pixels into two clean groups (dark group, bright group). "Best" means the biggest gap between the groups, measured as the **between-class variance**:

```
σ_between² = w₀ · w₁ · (μ₀ − μ₁)²
   w₀, w₁ = share of pixels in the dark / bright group
   μ₀, μ₁ = average brightness of each group
```

Used here to split **bright paper** from a **dark desk**.

- **Adaptive threshold**: *t* changes from place to place. For each pixel, compare it with the **average of the pixels around it** (minus a small constant). A pixel clearly darker than its neighbourhood is "ink". This survives uneven light (a shadow on one side of the page), where one global *t* would fail.

2.4 Morphology (cleaning masks)

Operations on white blobs in a mask, using a small shape (the "kernel", for example a 7×7 ellipse):

Operation	What it does	We use it to
Erode	shrink white blobs	(inside open)
Dilate	grow white blobs	smear text letters into a band
Open = erode then dilate	removes tiny white specks	ignore small bright dots when finding glare
Close = dilate then erode	fills small black holes and gaps	fill the "holes" the text makes in a white page, so the page is one solid blob

2.5 Connected components and contours

- **Connected components** (`connectedComponentsWithStats`): give each separate white blob a number and report its area and centre.

- **Contours** (`findContours`): trace the **outline** of each blob as a list of points. `contourArea` gives the area inside.
- **Convex hull** (`convexHull`): the outline you get by stretching a rubber band around the blob. Dents disappear.
- **approxPolyDP** (Douglas–Peucker): simplify a wiggly outline to a few straight corners, allowing it to move at most `eps` away from the original. Used to turn a page outline into **4 corners**.

2.6 Statistics on pixels

- **Mean** = average brightness. **Median** = the middle value (ignores freak outliers).
- **Variance** = how spread out numbers are: `var = average of (value − mean)²` . **Standard deviation** = `√variance` .
- **Median** shows up wherever one bad frame must not matter ("typical line height", "typical motion").

2.7 Projection profile

Add up a row or column of a mask to get one number per row/column. For a page of ink:

```
sum of ink along each row  →  a wavy line:
   high on text lines, low in the gaps between lines
```

This simple trick finds line spacing, line shifts, columns and paragraphs. It appears in dewarping, skew, spine and XY-cut.

2.8 Transformations

- **Affine** (2×3 matrix): move, rotate, scale, shear. Parallel lines stay parallel. (Used for rotation.)
- **Perspective / homography** (3×3 matrix): also changes the "viewing angle". Parallel lines can meet, like railway tracks. (Used to flatten a page photographed at an angle.)
- **Remap** (`cv2.remap`): for every output pixel, say *which input position to copy from*. Any shape of warp, even curved.
- **Interpolation**: the new position is rarely a whole pixel, so the colour is blended from the nearest pixels. `INTER_CUBIC` is the smooth, high-quality blend.

---

3. Part A: Reading the text (OCR)

OCR = Optical Character Recognition: turning a picture of letters into real text.

3.1 Apple Vision

`recognize_text()` hands the picture to **Apple Vision** (`VNRecognizeTextRequest`), a neural network that is part of macOS. It runs on this Mac's own chip, needs no download and no key, and sends the picture nowhere. We never see inside it; we only use what it returns.

Steps in the code:

1. Encode the picture to PNG (lossless) and give it to Vision.

2. Ask for the **accurate** level with **language correction** on, for English. (There is a "fast" level; it is not used because accurate already takes about 0.1 seconds and finds small print.)
3. For each line Vision finds, read back three things:
   - **text** (the letters),
   - **confidence** (0 to 1, how sure it is),
   - **box** (where the line is).
4. Sort lines top to bottom, left to right.

3.2 What a `TextLine` holds

Field	Meaning
`text`	the letters of the line
`confidence`	0–1
`box`	the axis-aligned rectangle around the line
`quad`	the **four real corners** of the line. When the page is slanted, the quad is slanted too, and `box` is only the straight rectangle that contains it
`words`	for each word: `(start, end, box)`, where start/end are positions in `text`
`word_quads`	the four corners of each word

Word positions need a little care. Vision counts characters in **UTF-16 units**, Python counts in code points; for ordinary English they are the same, but for rare symbols they differ, so the code converts.

3.3 `find_text_region()` — the quick "is there text?"

Used on live preview frames. Reads the frame, throws away lines with fewer than 2 letters/digits (stray marks), counts the characters left, and:

- fewer than **8 characters** → "no text";
- otherwise → one big box around all the text: `x1 = min of all left edges`, `y1 = min of all tops`, `x2 = max of all right edges`, `y2 = max of all bottoms`.

3.4 `read_text_enhanced()` — the careful read

For a photo you actually keep, we read **several versions** of the picture and keep the best one. Part B only offers extra versions when the frame needs them (see section 8). Each version gets a score:

```
score = Σ over lines of ( confidence × number_of_characters )
```

This rewards a version that is both **confident** and **recovers more text**. The untouched frame gets a **5 % head start** (`× 1.05`) so a fix only wins when it is clearly better.

---

4. Part B: Is the picture good enough? (quality)

`assess_quality()` produces a `QualityReport` with numbers and a list of problems. First the picture is turned into grey and shrunk to **640 pixels wide** (so the answer does not depend on the camera). Sharpness is measured **only inside the text box** when we know it, so the desk around the page does not count.

4.1 Sharpness (blur) — variance of the Laplacian

- A **sharp** photo of print has **sudden jumps** from black ink to white paper (strong edges).
- A **blurred** photo smears those jumps into gentle slopes.

The Laplacian kernel `[[0,1,0],[1,-4,1],[0,1,0]]` adds up the four neighbours and subtracts four times the middle pixel. In flat areas the answer is 0; at an edge it is large (positive on one side, negative on the other). Then:

```
sharpness = variance( Laplacian(grey image) )
```

- High variance → many strong edges → **sharp**.
- Low variance → few edges → **blurry**.
- `sharpness < 60` (`BLUR_MIN`) → problem **"blurry"**. The best number depends on the camera and how much text there is, so it can be set in `.env`.

4.2 Brightness and contrast

```
brightness = mean of grey pixels         (0 to 255)
contrast   = standard deviation of grey  (spread of brightness)
clipped    = share of pixels brighter than 245
```

Test	Problem
brightness < **50**	**dark**
brightness > **205** *and* clipped > **40 %**	**bright**
contrast < **25**	**flat** (ink and paper look alike)

4.3 Glare

Glare is a patch where light bounces straight into the camera and the page turns pure white, hiding the print. Reading *through* glare is impossible, so we find it and say where.

1. Mark pixels with grey **≥ 250** (nearly pure white).
2. **Open** the mask with a 7×7 ellipse. Plain white paper can reach 250, but it forms thin or speckled regions; real glare is a **compact solid patch**. Opening deletes the specks and keeps the patches.
3. `connectedComponentsWithStats` lists the blobs. The **biggest** one is the glare; its **centre** `(cx, cy)` is taken and divided by the picture size to give `glare_at`, a position from 0 to 1.
4. If the glare covers **≥ 1 %** of the picture (`GLARE_MIN_FRACTION`) → problem **"glare"**.

On real pharmacy photos this rule never fired (see section 20): the shine on foil and glossy cartons is bright but rarely reaches 250, so it is not "pure white".

4.4 Turning problems into one spoken hint

`hint()` picks the **most important** problem, in this order:

1. dark → "Low light"
2. glare → "Glare on the right. Tilt the page" (the side comes from `glare_at`: x > 0.62 right, x < 0.38 left, else y > 0.62 bottom, y < 0.38 top, else middle)
3. blurry → "Hold still, the image is blurry"
4. bright → "Too bright. Move away from the light"

---

5. Part B: Finding the page and flattening it (homography)

A page photographed at an angle looks like a trapezoid: the near edge is wide, the far edge is narrow, and the letters are squashed. OCR reads far better on a **flat, top-down** page. Two jobs: find the 4 corners, then undo the perspective.

5.1 `find_page_quad()` — find the 4 corners

There are two ways, and which one is used depends on whether the text has already been read.

When the text lines are known (a capture, and the live guidance): `find_item_quad()` grows the item from its own text.

1. **Pick the item's text.** The text lines are drawn as boxes, and boxes closer than **6 %** of the picture are joined into groups. The group with the most text wins; each line counts by its area (big print counts more) and by how near the middle of the picture it is (that is where people hold the item). Text on boxes in the background is a separate, smaller group, so it is ignored.
2. **GrabCut** (on a copy 320 px long): the text lines are marked "certainly the item", the area around them (up to 5 times the text block's size) "maybe", and everything further away "certainly background". The picture's rim is background too, except where it has the item's own colour (the colour under the text): an item that fills the picture reaches the rim. GrabCut then learns the colours of item and background and decides every "maybe" pixel. Because it compares colours, it works whether the item is **lighter or darker** than what is behind it.
3. Keep the piece that holds the text, and fill its holes (dark print, a barcode).
4. **Is it one solid four-sided item?** It must fill at least **85 %** of its convex hull, and the 4-corner shape (from `approxPolyDP`, or else the smallest rotated rectangle around it) must cover it with at most **25 %** mismatch. A shelf of several products or a crumpled strip fails, and then there is **no outline**: better none than a wrong one.

When the text is not known yet (a book page in book mode), the older rule is used. Paper is the **bright** thing on a darker desk:

1. Shrink the frame (longest side 480) and make it grey.
2. Gaussian blur 7×7 (kill noise).
3. **Otsu threshold** → white paper, black desk.
4. **Close** the mask with a square of about **4 %** of the picture size. The black letters make little holes in the white page; closing fills them, so the page is one solid white shape.

5. `findContours`, take the **biggest** blob. It must cover at least **25 %** of the picture, or there is "no clear page".
6. Take its **convex hull**, then `approxPolyDP` at several tolerances (2 %, 3 %, 5 %, 8 % of the outline length) until the outline simplifies to exactly **4 points**.
7. Safety check: the 4-point shape must fill at least **80 %** of the hull's area. A hand, a table edge or a cup is not rectangular enough, so it fails the check.
8. Scale the corners back up to the full picture size.

Why the text comes first. On the pharmacy photos the packs lay on a **light floor tile**. The floor was the brightest region, so the bright-region rule outlined the floor: 144 of its 145 outlines ran along the picture's own edges and none enclosed a pack. The same idea measured "how much of the picture the sheet fills" (`page_fill`), so the floor was counted as the item (median 69 % when the packs covered about 22 %), and on 26 photos the app would have stopped saying "Move closer" too early. Both now use the item grown from its text (corrected on 23 October 2026).

Ordering the corners (`_order_corners`) uses neat arithmetic on each corner `(x, y)`:

```
top-left     = smallest  x + y        (nearest the origin)
bottom-right = largest   x + y
top-right    = smallest  y − x
bottom-left  = largest   y − x
```

5.2 Does it even need flattening? (`_needs_warp`)

Warping a page that is already fine only adds blur. So we skip it when:

- **2 or more corners touch the frame edge** (the page runs out of the picture; warping would cut text off), or
- the page is nearly straight: tilt of the top/bottom edges **≤ 2°** *and* the **keystone** is **≤ 4 %**.

The keystone is how different opposite sides are:

```
keystone = max( |top − bottom| / max(top, bottom),
                |left − right| / max(left, right) )
```

(Equal opposite sides = a rectangle = no perspective.)

5.3 The homography (the maths)

A **homography** is a 3×3 matrix `H` that maps a point on the slanted page to its place on the flat page:

```
[x']       [h11 h12 h13] [x]
[y']  ∝    [h21 h22 h23] [y]
[w ]       [h31 h32 h33] [1]

final position = ( x'/w , y'/w )
```

The division by `w` is what creates perspective (things further away shrink). `H` has 8 free numbers (the ninth is a scale), and each point pair (slanted corner → flat corner) gives 2 equations. **4 corners × 2 equations = 8 equations**, which solves it exactly. That is why we need exactly 4 corners. OpenCV does this in `cv2.getPerspectiveTransform`.

The flat target is a rectangle whose size comes from the longest top/bottom edge for the width and the longest left/right edge for the height:

```
target = [[0, 0], [W−1, 0], [W−1, H−1], [0, H−1]]
```

`cv2.warpPerspective(..., INTER_CUBIC)` then builds the flat picture. (It must be at least 100×100.)

5.4 Going back

OCR finds boxes on the flat page, but the browser draws on the **camera** picture. So boxes are mapped back with the inverse matrix, H^{-1} (`cv2.perspectiveTransform` with `np.linalg.inv(matrix)`). The four corners of a box are moved, and the new box is the min/max of those moved corners (`unproject_box`).

---

6. Part B: Straightening curved lines (dewarp)

A thick book does not lie flat. Near the spine the paper curves, so the **lines of text bend** (they sag towards the spine). A homography cannot fix that because it only handles flat tilted planes. The project models the bend as: **every vertical column of the page is shifted up or down by some amount.**

```
shift(x) = how far the text lines have moved down at column x,
           compared with a flat part of the page
```

6.1 How the shift is found (`estimate_dewarp`)

1. **Grey, shrink** to at most 900 px, then make an **ink map** (`_ink`): an adaptive threshold that makes dark print = 1, paper = 0.
2. **Cut the page into 16 vertical strips.**
3. For each strip, add the ink along each row → a **row profile** (a wavy line: peaks on text lines). Blur it slightly.
4. The **weight** of a strip is the variance of its profile. A strip with real text has a strongly wavy profile (high weight); a margin strip is flat (low weight). Need at least 4 strips with decent weight, otherwise there is "too little text to model" and we do nothing.
5. **Line pitch** (distance between text lines): find it with **autocorrelation**, which means comparing the profile with copies of itself slid by different amounts. The first strong peak is the line spacing, because sliding by exactly one line makes the pattern match itself.

```
autocorrelation at lag k = Σ  p[i] × p[i + k]
```

6. Start at the **flattest/strongest strip** (the reference), then walk outwards left and right. For each strip, slide its profile against its **neighbour's** profile and keep the slide (`lag`) that matches best (**cross-**

correlation). A strip may only move up to **45 % of a line pitch**, so it can never jump to the wrong line. Shifts add up as we move outwards:

```
shift[strip i] = shift[neighbour] + lag
```

Weak strips just copy their neighbour.

7. If the biggest shift is tiny (under about 2 px), the page is flat: return nothing.
8. Fit a smooth curve through the shifts: a **degree-3 polynomial**, `shift(x) = a x³ + b x² + c x + d`, using **weighted least squares** (`np.polyfit`, weights = √strip weight, so reliable strips count more).
9. **Check it really helped.** Apply the shift to the shrunk image and compare "line crispness" before and after. Crispness is the **variance of the ink-per-row profile**: straight lines pile up ink into tall, clean peaks (high variance); bent lines smear it (low variance). The fix is kept only if crispness improves by at least **8 %** (`min_gain = 1.08`).
10. Re-express the curve in full-size pixels.

6.2 Applying it (`dewarp`)

```
output(x, y) = input(x, y + shift(x))
```

Done with `cv2.remap`: every output pixel copies from the input pixel `shift(x)` lower. After this, text lines run straight.

6.3 Going back (`unwarp_box`)

OCR boxes on the straightened image must be moved back onto the real camera frame: a box's top and bottom are moved by the shift at its left and right edges (the larger/smaller one), so the box still covers the bent text. For outlines (quads), every corner is moved point by point.

---

7. Part B: Fixing a slanted page (skew)

If the whole page is rotated a few degrees (not curved, just rotated), layout analysis (rows, columns, paragraphs) breaks, because it assumes lines are horizontal. So the code **measures the slant, straightens for analysis, then maps everything back** so the drawn outlines still follow the slanted text.

7.1 Measuring the angle (`estimate_skew`)

The idea: **when text is perfectly level, the ink sits on a few rows and the rows between are empty.** The row profile has tall peaks and deep valleys. When the text is tilted, the ink is spread over many rows and the profile is flat-ish. So:

1. Make an ink mask (adaptive threshold), shrunk to 500 px. If there is almost no ink, answer 0.
2. Only use a **circle in the middle** of the picture (so the empty corners that rotation creates do not distort the score).
3. For a trial angle θ: rotate the ink, sum each row, then score how "peaky" the profile is:

```
score(θ) = Σ ( row_sum[i+1] − row_sum[i] )²
```

Big jumps between neighbouring rows = sharp peaks = level lines.

4. Try θ from −25° to +25° in 1° steps (**coarse**), take the best, then try ±1° around it in 0.1° steps (**fine**).
5. Accept only if the angle is at least **1.5°** *and* the best score is at least **15 % better** than the score at 0°. Otherwise the page is "not slanted" and is left alone.

7.2 Rotating without cutting corners (`rotate_expand`)

A normal rotation clips the corners off. So the canvas is made bigger first:

```
new_width  = height × |sin θ| + width × |cos θ|
new_height = height × |cos θ| + width × |sin θ|
```

The rotation matrix gets an extra shift so the picture is centred on the bigger canvas. The 2×3 matrix is kept, and later `cv2.invertAffineTransform` brings every found box back to the original slant.

---

8. Part B: Fixing shadows and bad light

8.1 `enhance_tone()` — remove shadows, even out contrast

Geometry does not change (so OCR boxes stay valid). Steps:

1. Convert to **LAB** colour: `L` = lightness, `A` / `B` = colour. We fix only `L` so colours do not shift.
2. Estimate the **paper's brightness everywhere** ("the background"): shrink to a quarter, **dilate** with a big ellipse (dilation erases thin dark ink strokes, leaving the paper level), blur heavily, then enlarge back. This gives a smooth map of the light, including shadows.
3. **Divide** the image by this background:

```
flat = L / background × 245
```

Paper becomes about 245 everywhere, whether it was in shadow or in light; ink stays darker. A shadow is just "the paper is multiplied by 0.5", so dividing cancels it.

4. **CLAHE** (Contrast Limited Adaptive Histogram Equalisation, clip 2.0, 8×8 tiles): boost contrast **tile by tile**, with a limit so noise is not exaggerated.

8.2 `binarize()` — last resort

Adaptive Gaussian threshold: pure black ink on pure white paper. Aggressive, so it is only tried when the picture is bad but not blurry (blurred letters turn to blobs when binarised).

8.3 `capture_candidates()` — which versions to try

```
"original"                    always
"flattened"                   if the page needed a homography
"tone" / "flattened+tone"     if the quality report has any problem
"binarized"                   if there is a problem and the image is not blurry
```

`read_text_enhanced` reads each and keeps the best (see 3.4).

8.4 `prepare_page()` — for book pages

Same idea, but geometry must stay **invertible** (the reading highlight needs boxes in camera coordinates). So: dewarp the curved lines, tone-correct only if quality is poor, and return the image plus the dewarp model so boxes can be mapped back.

---

9. Part C: Book mode, when to read (page turns)

In book mode nobody presses a button. The camera watches the book, and when a new page appears and settles, it is read. `PageTurnDetector` does this in two ideas: **motion** and **layout fingerprint**.

9.1 Measuring motion (frame differencing)

Shrink each frame to **320×180**, grey, light blur. Then:

```
motion = average of | this_frame − previous_frame |      (cv2.absdiff, then mean)
```

If nothing moved, every pixel difference is near zero. A hand or turning page makes large differences.

Two guards against false alarms:

- **Median of the last 5 motions.** One noisy frame cannot start or stop a decision.
- **Adaptive noise floor.** Every camera has a different "still" level. The code keeps a running estimate with an **exponential moving average**, updated only on quiet frames:

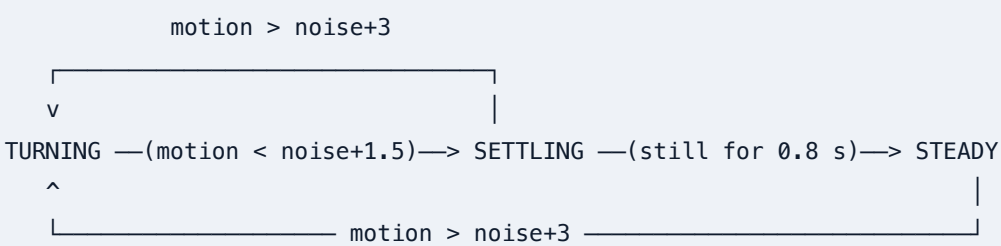
```
noise = 0.95 × noise + 0.05 × motion
```

Then movement is measured *relative to that floor*:

```
"moving"  when motion > noise + 3.0     (motion_on)
"still"   when motion < noise + 1.5     (motion_off)
```

Two different thresholds, with a gap between them, is called **hysteresis**. It stops the state from flickering back and forth when the value sits right on one line.

9.2 The state machine



- **TURNING**: something is moving (hand, page in the air).
- **SETTLING**: movement stopped; waiting to see it stays still.
- **STEADY**: still for `settle_time = 0.8` seconds → time to decide.

Between the two thresholds (not clearly moving, not clearly still) the "still" timer resets.

9.3 Is it a new page? The layout fingerprint

When the view becomes STEADY we must tell **a new page** from **the same page that merely shifted**. We compare a tiny fingerprint (`signature`) of the page layout:

1. Resize to 320×180, grey.
2. **Adaptive threshold** → ink = white (robust to lighting changes).
3. **Dilate with a wide 9×3 rectangle**: letters smear sideways into solid bands, so each text line becomes one band. Little shifts of the book now change the picture much less than a new page does.
4. Shrink to **80×45**.

Now compare two fingerprints with `layout_distance` : slide one over the other by up to **±4 cells** in each direction (81 tries), and for each, compute the average absolute difference (0 to 1). Keep the **smallest**.

```
distance = min over (dx, dy) in [−4, 4]²  of  mean | A_shifted − B_shifted | / 255
```

A book that shifted a bit lines up at some shift → small distance. A new page never lines up → large distance. If `distance < 0.06` (`change_min`), it is the same page.

9.4 The events

- **"changed"**: view is steady **and** the layout differs from the last page read (a new page, or the first page).
- **"moved"**: view is steady after movement **but** the layout looks the same. A hand passed over, the book shifted, or a very similar-looking page was turned (dense pages look alike), so the caller must compare the *words* to decide.
- While STEADY, the layout is re-checked once a second, because a slow drift never looks like motion.

---

10. Part C: Book mode, where the pages are (the spine)

An open book is a **two-page spread**. The two pages must be read separately (left, then right), so we need the **gutter** (the spine, the valley in the middle). `find_spine()` reports the x position of the spine, or `None` for a single page. First it finds the **book's border** (10.0); then it looks for evidence of the spine inside it

(`split_spread()`, steps 10.1 to 10.5); and when there is none, the spine is the **centre of the border**, because the spine of an open book is nearly always there.

10.0 The book's border (`find_book_quad`)

The whole frame is read once (Apple Vision), then the open book's outline is found as **four corners that follow its tilt**:

1. **Apple's own document detector** (`apple_document_quad`): `VNDetectDocumentSegmentationRequest`, the same one the Files and Notes apps use to outline a sheet when you scan it. It takes about 0.07 s. When it is unsure it can return a sliver or corners that cross over, so its outline is used only when it is convex, every corner is between 60° and 120°, and it holds at least 90 % of the book's text (the centres of the text lines). An outline round only one page of a spread fails that test, which matters: a tilted book is flattened by cropping to its border, so the other page would be lost.
2. **Otherwise, grown from the text** (`find_item_quad(..., every_piece=True)`): GrabCut starts from the text lines (certainly paper) and grows out by colour, as for a medicine pack (section on the item outline). For a book every piece that holds text is kept, since the fold can cut the paper in two, and the four corners are fitted round all of them.

"The book's text" is the group of lines lying within **30 % of the picture** of each other (`BOOK_GAP`), so both pages count, even an index page whose short entries leave a wide blank before the facing page, while text on things lying around the book does not. Neither way takes a light or patterned cloth for paper; the brightness trick of 10.1 does. On the 5 real book photos (on a yellow bedspread) Apple's detector gave the border on 3 and the text-grown outline on the other 2.

When the border is found: a tilted book is first **flattened by its own four corners** (homography, as in Part B), and the border, not 10.1, decides whether it is a spread (10.2). On screen the border is drawn in **red** and the spine in **yellow**, as a line from the border's top side to its bottom side (`spine_line`) that leans with the book.

10.1 Step 1: find the book (`find_book`, only when there is no border)

Same trick as the page finder: blur (21×21), **Otsu** (paper is bright), **close** with a 25×25 square to fill the text, take all big bright blobs (each at least 5 % of the picture, together at least 20 %), and return the **bounding box of them all**. A spine shadow can cut the paper into two halves, which is why it uses all the big blobs together. If nothing is found, use the whole frame.

10.2 Step 2: is it even a spread?

A two-page spread is clearly **wider than tall**. If `book_width < 1.1 × book_height` (`SPREAD_MIN`), it is one page: return `None`. With a border, its width and height are the means of its opposite sides.

10.3 Step 3: clue 1 — the spine's shadow

A half-open book curves down into the spine and casts a **dark valley**. Take the average brightness of each column (rows 10 %–90 %), smooth it, and look for the darkest column in the middle band (30 %–70 % of the book width):

```
valley found  if  brightness at the dip  <  (1 − 0.12) × the smaller of the
                  typical brightness on the left and on the right
```

(`min_depth = 0.12`, a dip of at least 12 %.)

10.4 Step 4: clue 2 — the smooth, text-free strip

If the book lies flat and fully open, there is no shadow. But **text never crosses the spine**, so the two inner margins form a **smooth, empty vertical strip**. To detect "smooth", compute **texture**:

```
local std  =  sqrt( mean(x²) − (mean(x))² )          over a 15×15 window
```

(That is the variance shortcut `E[x²] − E[x]²`.) Average it down every column (`texture_profile`). Text is "busy" (high std even when blurred); blank paper and the spine are smooth (low std). A column is **quiet** if its texture is below 60 % of the typical text level. Runs of quiet columns that are at least **7 % of the book width** wide (wider than the gap between two *columns of text* on one page) are candidates. Pick the one nearest the middle and put the spine at its centre.

10.5 Step 5: the sanity check — text on both sides

Either clue only counts if **both sides** of the proposed spine hold a real block of text (at least **20 % of the book width** of busy columns each side). This stops three false alarms:

- a single wide sheet,
- a page with two columns (the gap between columns looks like a spine),
- a spread with text only on one side.

If neither clue passes and there is a border: the spine is the **centre of the border** (the mean x of its four corners). If there is no border either: `None`, a single page, and no line is drawn.

---

11. Part C: Page number, title, header, footer

Printed page numbers and running titles ("THE SILENT RIVER") should be *announced*, not read as if they were story text. `split_page_parts()` separates them from the body using only the **positions and sizes** of the OCR lines.

11.1 Group lines into rows

`_rows()` puts lines that sit side by side (their vertical ranges overlap) into one row. For example, a footer "46 ... THE SILENT RIVER" may be OCR'd as two lines on the same row.

11.2 What counts as a header or footer row

Only the **very first** or **very last** row can be one, and it must be **set apart** from the body:

```
gap to the next row  >  max( 1.8 × usual_gap , 0.9 × line_height )
```

(`usual_gap` is the median gap between rows; `line_height` is the median line height.) And it must be either **short** (narrower than 60 % of the usual row width) or **contain a page number**. At least 3 rows are required, otherwise there is nothing to compare.

11.3 Reading the page number (`parse_page_number`)

Regular expressions understand: `47`, `– 47 –`, `Page 47`, Roman numerals like `xii` (a Roman-numeral pattern, up to 7 characters), `47 THE SILENT RIVER` (number then title) and `CHAPTER THREE 48` (title then number). Result: `(number, title)`.

11.4 Headings

A body line is a **heading** if it is clearly taller than normal (`≥ 1.35 ×` the median line height) or it starts with a word like *chapter, part, section, preface, introduction, contents, appendix*.

11.5 Missing numbers on a spread

If one page of a spread shows a number and the other does not, the other is **inferred**: the left page is the right number minus one; the right page is the left number plus one. `label()` then gives "Page 47" or "Pages 46 and 47".

---

12. Part C: In what order to read (XY-cut)

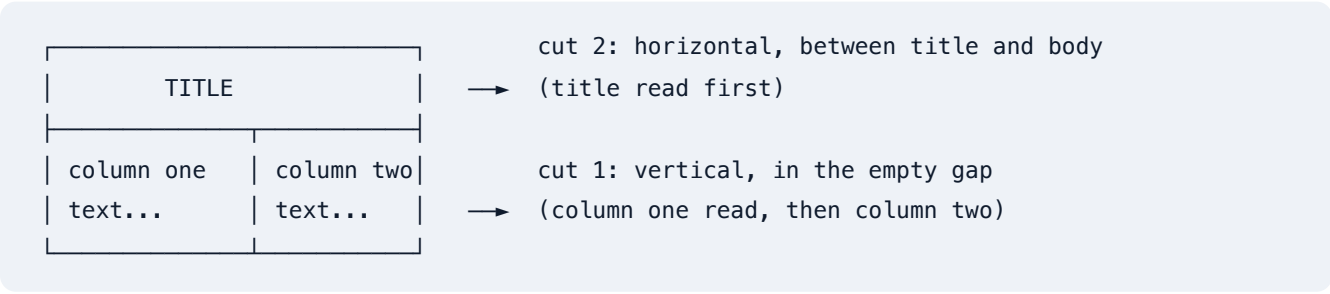
OCR gives lines in a heap. A newspaper page with two columns must be read the whole left column first, then the right, not line by line across. **XY-cut** is a classic layout algorithm that finds that order using only the **empty gaps** between the text boxes. It is "recursive": cut the region in two, then cut each piece again.

12.1 The rule

For a group of lines:

1. **Vertical cut (columns).** Project every line box onto the **x axis** and merge overlapping spans. An empty band **at least 1.2 × line height wide** is a column gap. Cut at the middle of the **widest** such gap → left group and right group. Read left first. The column boxes are also recorded (for drawing).
2. **Otherwise, horizontal cut (headings / paragraphs).** Project onto the **y axis**. A gap clearly bigger than normal line spacing, at least `max(1.8 × usual_gap, usual_gap + 0.6 × line_height)`, splits the group into upper and lower. Read the upper first.
3. **Repeat** on each piece.
4. **Nothing left to cut?** Read the group line by line from the top. A line that is **indented** (starts more than `max(0.6 × line_height, 6 px)` right of the usual left edge) starts a **new paragraph** (novel style).

12.2 A tiny picture



Why this works: it needs no learning, only "*where is there nothing?*". The OCR boxes are accurate whenever the text is readable, so the gaps are reliable.

13. Part C: Putting the whole page together

`read_page()` does all of Part C for one camera frame. In order:

1. **(If `enhance`) flatten** the frame with the homography if the page is tilted. If the flat version reads nothing, fall back to the frame as the camera saw it.
2. **Find the book's border and the spine** → flatten a tilted book by its border, split into left and right page crops.
3. For each page crop:
   - measure and fix **skew** (`estimate_skew`, `rotate_expand`);
   - **dewarp** curved lines and fix tone (`prepare_page`);
   - **OCR with word boxes**;
   - map lines back through the dewarp (`_unwarp`);
   - split off **header/footer** and read **page number / title / headings**;
   - **XY-cut** the body into paragraphs in reading order;
   - map every box back to full-frame pixels (`_page_back`, which undoes the rotation with the inverse affine matrix and adds the page's x offset).
4. **Build the text.** Paragraphs are separated by a blank line. Lines inside a paragraph are joined with a space, **except** a line ending with a letter and `-` followed by a lowercase start is merged into one word: `"exam-" + "ple" → "example"`. A dash after a space or a sign is kept (`"Career Progression –"`, `"Rs. 400/-"`).
5. **Remember where everything is.** Each printed line records `(start, end)`, its character range in the final text, plus its box and its word boxes. This is what lets the screen **highlight the exact word being spoken**: speech reports "I'm at character 340", and the app finds the word whose range contains 340.

The result is `(text, PageLayout)`, where `PageLayout` holds the spine x, columns, paragraph blocks, margins, line spans, paragraph ranges, and each page's number/title/headings. All of it is in **camera pixel coordinates**; `layout_json` converts it to 0–1 for the browser.

13b. Paragraphs by the language model (`arrange_page`)

The XY-cut cannot read the meaning, so it makes false paragraph breaks: a sentence that runs from the bottom of one column to the top of the next, or over a page, looks like a new paragraph, and every paragraph break is a pause in the speech. The text reader also misreads letters (`rn` for `m`, `l` for `I`, `vv` for `w`), which a picture alone cannot catch but meaning can. So before a new page is spoken, the language model gets the lines, decides **which lines go together**, and **reads the page for sense**, correcting lines the reader got wrong.

0. `line_blocks()` first groups the lines into **blocks** using only the boxes: two lines are in the same block when one sits right under the other (a gap of at most 1.2 line heights), with the same left edge (within 2 line heights) and mostly the same horizontal span (at least 60 % overlap). This is a union-find over all line pairs. A column, a heading and a **sidebar** each become a block, so a sidebar whose rows are level with the main text's rows is no longer mixed into it. (Plain XY-cut cannot do this: there is no empty gap that runs across the page, so it falls back to reading row by row and alternates between the two.)
1. `line_hints()` adds a short tag to each line from its box: `[page]` (first line of a new page of a spread), `[column]` (the line starts above where the previous one ended), `[gap]` (more than 0.8 line heights of

space above), `[indent]` (starts more than 0.6 line heights right of the column's usual left edge), `[short]` (ends more than 3 line heights before the column's right edge, the 80th percentile of the right edges).

2. `numbered_lines()` shows the model the lines block by block, one row per line, with the line number, where it is and how big its letters are (`x62–98 y55 h0.7`: from 62 % to 98 % of the text area's width, 55 % of the way down, letters 0.7 times the usual size, so small print marks a sidebar) and the tags: `3 (x78–100 y35 h0.9)[column][indent] Later that day...`.
3. The model replies with JSON only: `{"paragraphs": [[0,1,2],[3,4]], "skip": [5], "fixes": {"3": "Later that day the rain began"}}`: the paragraphs **in reading order** (the left page first; on a page all the main text, then each sidebar), each a list of line numbers, which need not be consecutive, and the corrected lines. A paragraph may join lines of two blocks when a sentence runs on from one into the other. A fix may only repair what the reader got wrong (letters, a split or joined word, a stray symbol): never add, drop or reword, never correct the author, and leave a line alone when unsure.
4. `parse_arrangement()` checks it: no line number may appear **twice** or be off the page, no paragraph may be empty, and no more than a quarter of the page may be skipped; a reply that fails is thrown away. A line the model **forgot** (on real spreads of 110 to 170 lines it often drops a few, once 20 in a row) is put back right after the nearest earlier line, which is where the layout reads it, so nothing is lost and the rest of the answer is still used. Fixes are kept only for lines that are on the page and read.

   4b. `fix_is_safe()` then checks each fix against the line as read: at least 60 % of the letters must match (`difflib.SequenceMatcher`), the length may change by at most about 30 %, and the word count by at most a quarter. A fix that rewrites or adds to a line is refused and the line is read as the camera saw it. On the 5 real book photos the model's fixes were taken 64 times and refused once (it tried to add words not in the picture). Most were clear (`chargos` → `charges`, `emplovee on transter` → `employee on transfer`); some fill in letters cut off at the page edge (`notion` → `Stagnation`), which is a good guess rather than a certainty.
5. `regroup()` rebuilds the text from the lines, with the accepted fixes in place (paragraphs joined by a blank line, lines by a space, hyphenated words joined), and recomputes every character position. A corrected line's words take the boxes of the read words at the same place in the line, so the word highlight still lands on them. The paragraph boxes are rebuilt too (a rotated rectangle around the lines, found with `cv2.minAreaRect`).

If there is no key, the model is slow (12 s), the page has more than 250 lines, or the reply fails the check, nothing changes and the page is read with the XY-cut paragraphs and the words as read. The corrected text is also what the model answers questions from. With the local Qwen model this step usually times out on real book pages (their prompts are 4,000 to 6,000 tokens, more than Qwen's 4,096-token window), so the words are checked only with Gemini. The result is remembered by the page's lines, so the same page is not sent twice.

---

14. Part C: Carrying on after the view moves

If the view moves (a hand passes, the phone is nudged), the app should **not restart the page from the first word**. It should compare the new text with the old and pick up where it was.

14.1 Matching words (`SequenceMatcher`)

Both texts are cut into words, lower-cased, punctuation removed. Python's `difflib.SequenceMatcher` finds the **longest matching runs of words** in the same order (the Ratcliff–Obershelp method). Only runs of **at least 3 words** count, so common words like "the" and "and" on a *different* page do not look like a match.

```
overlap = matched words / words in the shorter text
```

If `overlap < 0.4` (`min_overlap`) → **different page** (`None`).

14.2 Where to carry on

`old_pos` is how far reading had got (a character position). Find the first old word not finished yet (`reached`). Then:

1. If that word is still **in view** (inside a matching run) → carry on from the **same word** in the new text.
2. Otherwise → carry on from the first word **after** the part already read.
3. If reading had not yet reached the visible part → start at the first visible matched word.

14.3 A second, gentler check

A moved view with heavy blur may be too garbled to line up. `word_overlap` (the same measure) with a looser limit (`MOVED_SAME_PAGE = 0.12`) says "still the same page, keep going; never restart".

---

15. Part D: Live guidance

`LiveGuide` is the coach for normal (non-book) mode. It is fed preview frames; each frame gives back what to draw and what to say, and when to take the full-size photo.

15.1 The guide box

A rectangle inset slightly from the picture edges (`30/1280` and `20/720` of the width/height). The text must be inside it.

15.2 How much of the text is inside the box

```
inside = area( text box ∩ guide box ) / area( text box )
```

The overlap rectangle is: left = max of lefts, top = max of tops, right = min of rights, bottom = min of bottoms; if that has no width or height, the share is 0.

15.3 Position hints (`_guidance`)

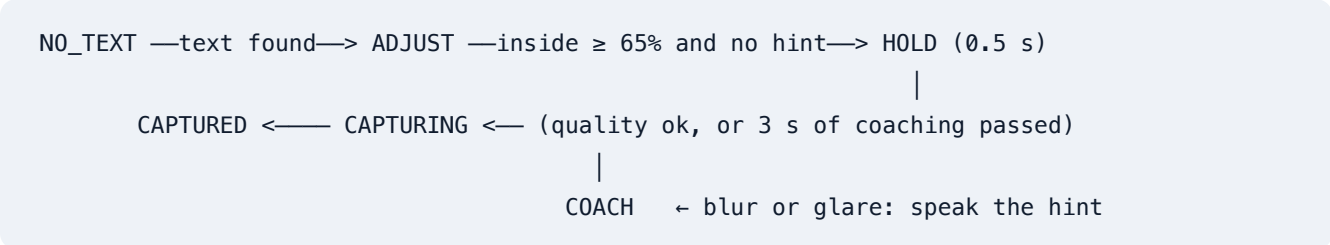
All distances scale with `height / 720`, so they work at any resolution.

1. **Too small and not read well?** If the **median line height** is under `14 × h/720` pixels **and** the OCR is not sure of the text → **"Move closer"**. (Median ignores a few odd lines. "Sure" means the mean confidence of the lines, each counted by its letters, is at least `READ_WELL = 0.8`; small text that is read well gets no hint and goes on to HOLD and capture. A third test keeps the hint away when **the item**

already fills the picture: `page_fill()` is given the text lines the coach just found and measures the area of the item that carries them, using the same item mask as the outline (`item_mask()` : GrabCut grown from the text, section 5.1); if that is at least `PAGE_FILLS = 0.75` of the frame, a page, an open book, a pack or a lit tablet screen fills the view and is close enough. GrabCut costs about 0.1 to 0.25 s on a 960 px preview frame, so it is only computed when the other two tests would have said "Move closer". Without text lines `page_fill()` falls back to the old rough rule: Otsu threshold on a 320 px copy, print closed away with a 9×9 square, big bright regions (each over 3 % of the frame) added up, about 1 ms, but it counts a light floor as paper.)

2. **Touching an edge and far off-centre?** The text box touches the guide box edge (within 5 px) and its centre is more than 80 px from the guide's centre:
   - touching left/right → **"Move Left"** or **"Move Right"** (the hint says which way the *page* should move so the picture is centred);
   - touching top/bottom → **"Move Up"** or **"Move Down"**.

15.4 The capture state machine



- **HOLD**: the text has to stay inside the box for **0.5 s** (`HOLD_TIME`), so a passing glance does not trigger a photo.
- **COACH**: if the frame is **blurry** or has **glare**, coach for up to **3 s** (`QUALITY_PATIENCE`); if it does not improve, take the photo anyway (Part B's extra versions then try to rescue it).
- **CAPTURING**: tell the browser to take the full-size photo.
- **CAPTURED**: stay quiet until the item **leaves the view for 1.5 s** (`REARM_AFTER`), then be ready for the next one.
- **Not too chatty**: the same position hint repeats only after 2 s; a quality hint at most every 5 s; "Low light" at most every 30 s.

---

16. Part D: The book-mode decision

`BookWatcher` feeds frames to `PageTurnDetector` . When the view is steady, `BookReader.process()` reads the page (`read_page(..., enhance=True)`) and chooses what the browser should do:

Action	When	What the browser does
`new`	a different page	(the model's paragraphs are applied first) say "Page 47." (and the running title if it changed), then read all paragraphs from the start
`resume`	same words in view, but the view moved	read from the word reached

Action	When	What the browser does
`keep`	a hand passed over while reading, or the page is the same but too garbled to match	change nothing, keep reading
`nothing_new`	same page, nothing left to read	stay quiet
`no_text`	nothing readable	stay quiet

Extra rule: **different printed page numbers always mean a new page**, even if the words look similar.

For speech, `speech_chunks()` cuts the text into pieces: one per paragraph, and a very long paragraph (over 600 characters, or 300 when the local Kokoro voice is used, so each piece is made in a few seconds) is cut at the last `.`, `?` or `!`, or at a space if there is no sentence end. Each piece is spoken as its own utterance, and its start position is known, which is what powers the word highlight.

---

17. The glare experiment (glare.py)

The real-photo test in section 20 showed why this matters: the app's pure-white rule never caught the shine on foil and glossy packs, which is bright but not saturated.

`glare.py` was a standalone test script, kept outside this repository (it is not in the project and not used by the app). It detects glare in a different way from `assess_quality`, and draws red circles on the picture.

1. Convert to **HSV** colour: **H**ue (which colour), **S**aturation (how colourful), **V**alue (how bright).
2. Glare = **very bright (V > 235) AND nearly colourless (S < 40)**. Glare washes colour out to white, so its saturation drops to almost 0. This separates glare from a bright but coloured area, such as a yellow page.
3. **Close** the mask (7×7 ellipse) to join nearby fragments.
4. `findContours`, and for each patch `minEnclosingCircle` (the smallest circle that holds it).
5. Ignore circles with a radius of 10 pixels or less (noise). Draw a red circle on the others.

Comparison with the app's own glare test:

	`assess_quality` (in the app)	`glare.py` (experiment)
Colour space	grey	HSV
Rule	grey ≥ 250	V > 235 and S < 40
Cleaning	**open** (delete specks)	**close** (join fragments)
Output	centre → spoken hint	circles drawn on the picture

---

18. Cheat sheet: every number and why

Number	Where	Meaning
640 px wide	`_ANALYSIS_WIDTH`	quality is measured at a fixed size so cameras compare fairly
sharpness **< 60**	`BLUR_MIN`	blurry
brightness **< 50**	`DARK_MEAN`	too dark
brightness **> 205** and clipped **> 40 %**	`BRIGHT_MEAN`	too bright
contrast **< 25**	`assess_quality`	flat, ink and paper too alike
grey **≥ 250**, area **≥ 1 %**	`GLARE_MIN_FRACTION`	glare
glare side split **0.38 / 0.62**	`hint()`	left/right/top/bottom/middle
text groups joined within **6 %** of the picture	`ITEM_GAP`	the item's own text, not the background's
item may grow **5 ×** past its text	`ITEM_GROW`	room for the whole pack, not the room
item fills **≥ 85 %** of its hull, quad mismatch **≤ 25 %**	`ITEM_SOLID` , `ITEM_FIT`	one solid four-sided item, or no outline
page area **≥ 25 %**	`find_page_quad`	there is a clear page
approxPolyDP **2/3/5/8 %**	`find_page_quad`	tolerance to reach 4 corners
quad area **≥ 80 %** of hull	`find_page_quad`	really rectangular
tilt **> 2°** or keystone **> 4 %**	`_needs_warp`	page needs flattening
16 strips, degree 3	`estimate_dewarp`	detail of the bend model
max shift **45 %** of line pitch	`estimate_dewarp`	never jump to a wrong line
gain **≥ 1.08**	`estimate_dewarp`	the fix must make lines 8 % crisper
skew range **±25°**, steps 1° then 0.1°	`estimate_skew`	search for the level angle
skew **≥ 1.5°** and **+15 %** better	`estimate_skew`	only fix real slant
CLAHE clip **2.0**, 8×8 tiles	`enhance_tone`	local contrast
320×180 frames	`PageTurnDetector`	motion is measured on a small picture
median of **5**	motion	one noisy frame is ignored
noise `0.95/0.05`	motion	slow learning of the camera's still level
moving **noise + 3**, still **noise + 1.5**	hysteresis	no flicker
settle **0.8 s**	`settle_time`	page must be still before reading
fingerprint **80×45**, dilate **9×3**	`signature`	compact layout "fingerprint"
shift **±4** cells, distance **< 0.06**	`layout_distance`	same page, only moved
lines within **30 %** of the picture	`find_book_quad` (`BOOK_GAP`)	the book's text: both pages

Number	Where	Meaning
corners **60°–120°**, holds **≥ 90 %** of the text	`apple_document_quad`, `find_book_quad`	Apple's border is believable
book aspect **> 1.1**	`find_spine`, `split_spread` (`SPREAD_MIN`)	a spread is wider than tall
shadow dip **≥ 12 %**	`split_spread`	spine shadow
quiet strip **≥ 7 %** wide, texture **< 60 %**	`split_spread`	text-free strip
text on each side **≥ 20 %**	`split_spread`	both pages hold text
heading **≥ 1.35 ×** line height	`split_page_parts`	larger type = heading
column gap **≥ 1.2 ×** line height	`xy_cut`	split into columns
paragraph indent **> max(0.6 × line h, 6 px)**	`xy_cut`	new paragraph
word run **≥ 3**, overlap **≥ 0.4**	`continue_from`	same page
same page when overlap **≥ 0.12**	`MOVED_SAME_PAGE`	too garbled to line up but same page
text height **< 14 px (of 720)**, mean confidence **< 0.8**, and the sheet covers **< 75 %** of the frame	`MIN_TEXT_HEIGHT`, `READ_WELL`, `PAGE_FILLS`	"Move closer" (small text that is read well, or a sheet that fills the picture, needs no hint)
inside guide **≥ 65 %**	`LiveGuide`	text in view
hold **0.5 s**, coach **3 s**, rearm **1.5 s**	`LiveGuide`	timing of capture
speech chunk **≤ 600** characters	`speech_chunks`	one utterance

19. Math glossary

Word	In plain words
Pixel	one square dot of a picture, holding a number (or three)
BGR / RGB	the order of the three colour numbers; OpenCV uses BGR
Greyscale	one brightness number per pixel
Kernel	a small grid of weights slid over a picture
Convolution	at each spot, multiply the picture by the kernel and add up
Gaussian blur	average with neighbours, nearer ones count more (bell-curve weights)
Laplacian	edge detector; its variance measures sharpness
Variance / std	how spread out numbers are (`std = √variance`)
Median	the middle number; ignores freak values
Threshold	turn grey into black/white at a cut-off

Word	In plain words
Otsu	threshold chosen automatically to best split two groups
Adaptive threshold	cut-off that changes from place to place
Mask	a black/white map of "interesting pixels"
Morphology	grow, shrink, open, close on white blobs
Contour	the outline of a blob
Convex hull	the rubber band around a blob
approxPolyDP	simplify an outline to a few straight corners
Homography	3×3 matrix that flattens a perspective view
Affine transform	move, rotate, scale (2×3 matrix)
Interpolation	blending nearby pixels when a position falls between pixels
Remap	for each output pixel, copy from any chosen input position
Projection profile	adding a row/column of a mask into one number per row/column
Autocorrelation	comparing a signal with a slid copy of itself; peaks show repeat spacing
Cross-correlation	comparing two signals at different slides; the peak shows the best alignment
Polynomial fit	a smooth curve `a x³ + b x² + c x + d` through points (least squares)
Weighted least squares	a curve fit that trusts some points more
Frame differencing	motion = average of `|frame − previous frame|`
Exponential moving average	`new = 0.95 × old + 0.05 × measurement`; a slow-learning average
Hysteresis	two different thresholds (on and off) so a state does not flicker
State machine	a small set of states (TURNING, SETTLING, STEADY) and rules for moving between them
XY-cut	recursively cut the page at empty gaps, along x for columns and along y for paragraphs
Quad	four corners; a slanted rectangle
Normalised coordinates	positions as fractions (0–1) of the picture size
OCR	turning a picture of letters into text
CLAHE	local contrast boost with a limit on noise
LAB colour	a colour space where `L` is brightness alone
HSV colour	hue, saturation, value (brightness)
Sequence matching	finding the longest runs of matching words between two texts

20. How the checks did on 148 real photos

On 22 October 2026 every check above was run on 148 phone photos from a pharmacy: tablet strips, cartons, syrup bottles, ampoules, tubes and jars (`eval/run_photo_eval.py`). Each photo was saved with every check drawn on it as a box of its own colour, so you can **see** what the app found, not just read numbers:

Colour	What the box shows
Green	each line of text Apple Vision found (its slanted outline)
Cyan	the text region: one box around all the text (what live guidance aims at)
Yellow	the item outline from `find_page_quad()` (grown from its text)
Magenta	each saturated glare blob (grey 250+, after the 7×7 opening)
Red	a 40 px patch on a text line whose sharpness (variance of the Laplacian) is below 60
Blue	a 40 px patch darker than the "dark" level (mean below 50)
Orange arrow	the text slant from `estimate_skew()`

What came out:

Check	Result	What it means
Blur (4.1)	Text sharpness 248 to 7934, median 1966; never below 60. 21 of 8466 small text patches were blurry, on 13 photos	Phone photos held still are far above the threshold; blur was not a problem in this set
Glare (4.3)	Never flagged. The largest saturated blob was 0.12 % of the picture (the rule needs 1 %). Blobs were drawn on 15 photos, only 6 of the 44 photos with visible glare	Shine on foil, gold cartons and glossy bottles is bright but not pure white; a mirror-like carton even reflects a *dark* image. A better glare test would look for bright, low-texture patches **on the text region**, not only pure white
Light and contrast (4.2)	Brightness 96 to 167, contrast 34 to 70; never "dark", "bright" or "flat"	The photos were taken in a lit shop. The blue "dark" patches are dark printing (navy labels, black bands), not poor light
Slant (7)	Measured on 64 photos, median 3°, up to 26°	Only the book path straightens slant; a single capture is read as it is (Apple Vision copes with moderate slant)
Item outline (5)	First version: an "outline" on 145 photos, but 144 were the picture's own frame and none enclosed a pack. Corrected: an outline around the item on 134 photos, none on shelves of several products or crumpled strips; 71 tilted enough to flatten	The bright-region rule assumed white paper on a dark desk; on a light floor it found the floor. Growing the item from its text fixed it
Item fill (15)	First version: median 69 %, 47 photos at 75 % or more. Corrected: median 22 %, none at 75 %	The floor had been counted as the item, which would have stopped "Move closer" on 26 photos where the pack was small

Check	Result	What it means
Capture versions (3.4)	Corrected outline: the untouched photo on 134, the flattened item on 14	With a real outline a tilted item is straightened before reading; the text the app keeps improved slightly (90.1 % against 89.7 % key-line accuracy), and the answers stayed the same (62.8 %)

To look at one check at a time, `eval/make_check_folders.py` puts the photos into `eval/data/private/cv_checks/blur_photos`, `glare_photos`, `dark_photos`, `slant_photos` and `outline_photos` (13, 15, 126, 64 and 134 photos), each with only that check's boxes. The full table, and every photo with its boxes, is in eval_photo_summary.md and `eval_photo_report.md` (kept out of git because it holds the photos' text).

On 28 October 2026 the photo test was run again on **164 photos** (the 148 above plus 16 food labels, book pages, documents and screens) with the current code and a fixed evaluation date: the app's capture text reached 90.5 % key-line accuracy (character error rate 0.095), and the glare check still never fired. See eval_results.md.

---

21. Book mode and live checks on real input

Measured on 28 October 2026 (`eval/run_book.py`, `eval/run_latency.py`); the numbers are in eval_results.md.

Five photos of a real book (4 of them two-page spreads), read with book mode's path:

Step	Result	What it means
Spine split (10)	0 of 4 spreads split	Neither clue fired on these photos, so each spread was read as one wide page. On the synthetic spreads used in the tests the split works (6 / 6)
Page numbers (11)	0 of 5 announced	With no split, the printed page numbers sit inside the body text of one wide "page", so `page_numbers()` finds none
Reading order (12, XY-cut)	98.4 % of key-line pairs in the right order (309 of 314); reading started at the first body line on 4 of 5	Even unsplit, the layout order follows the left page and then the right page
Paragraph grouping by the model (13)	0 of 5 pages within the 12-second limit (Qwen 3B)	Dense pages have too many lines for the small model in time; the page is then read in layout order, as designed

A 30-second phone video of 4 page turns, fed to `BookWatcher` at 8 frames a second (the rate the page sends live frames): 6 "changed" events for 5 pages. Every turn was caught; looking at the saved frame of each event, the extra one came at 17.3 s while the hand was still over the page in the middle of a turn, so book mode would have started reading a half-covered page. No "moved" events.

Live checks. A live frame (960 px) takes 0.11 s on the server (it could take about 9.3 a second), but the page waits 100 ms after each reply before sending the next frame, so the checks run about **4.8 times a second**, below the requirement's 10. A book-mode frame (page-turn watch only) takes 0.004 s.